EXPERIMENT – 3

Aim: Apply Decision Tree and Random Forest for Classification Tasks.

Theory:

1. Dataset Source

- Dataset Name: **ADANIPORTS.csv**
- Source: NSE Historical Stock Market Data
- Kaggle Link: https://www.kaggle.com/datasets/rohanrao/nifty50-stock-market-data
- Domain: Financial Analytics / Stock Market Prediction

The dataset contains daily historical trading data of Adani Ports including price variations and trading volume.

---

2. Dataset Description

The dataset consists of daily stock trading records. Since Decision Tree and Random Forest are classification algorithms, the dataset was converted into a classification problem.

Dataset Characteristics

- Type: Structured tabular dataset
- Nature: Time-series financial data
- Features Used: 4 numerical attributes
- Task: Binary classification

Features (Independent Variables)

Feature	Description

Open	Opening price
High	Highest price during the day
Low	Lowest price during the day
Volume	Number of shares traded

Target Variable (Created)

A new feature called **Movement** was created:

$Movement$={1if Close > Open (Stock Up)0otherwise (Stock Down)Movement={10if Close > Open (Stock Up)otherwise (Stock Down)

This converts the regression dataset into a binary classification dataset.

---

3. Mathematical Formulation of the Algorithms

3.1 Decision Tree (Classification)

Decision Tree splits the dataset based on feature values to reduce impurity.

Gini Index (Used for splitting)

$Gini$=1−∑i=1cpi

Where:

- pipi = probability of class i
- c = number of classes

The algorithm chooses the feature and threshold that minimizes Gini impurity.

Alternatively, entropy can be used:

$Entropy$=−∑i=1cpilog2(pi)

The tree continues splitting until stopping criteria such as max depth are met.

3.2 Random Forest

Random Forest is an ensemble learning method that builds multiple decision trees.

Each tree is trained on:

- Bootstrap sampled dataset
- Random subset of features

Final prediction:

$\hat{y} = \text{Majority Vote}(T_1, T_2, \ldots, T_n)$

This reduces variance and improves model generalization.

---

4. Algorithm Limitations

Decision Tree

- Prone to overfitting
- High variance
- Sensitive to small changes in data
- Can create very complex trees

Random Forest

- Computationally expensive
- Less interpretable than a single tree
- Requires tuning of number of trees

Stock market data is noisy and volatile, which affects classification accuracy.

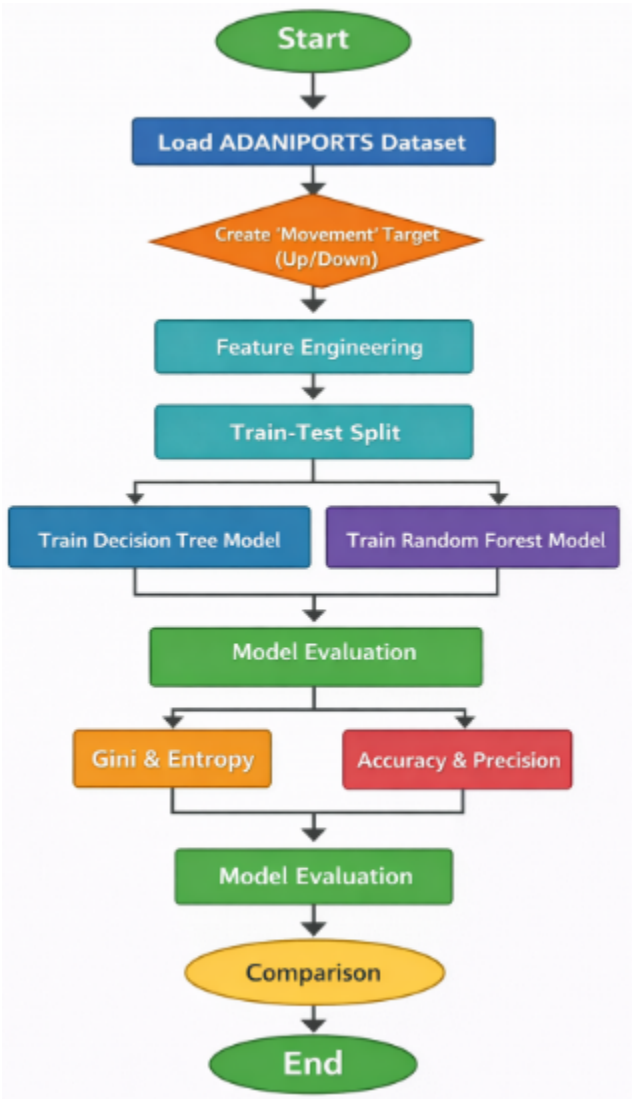
---

5. Methodology / Workflow

Step 1: Load ADANIPORTS dataset
Step 2: Inspect data and check for missing values
Step 3: Create Movement classification target

Step 4: Select features (Open, High, Low, Volume)
Step 5: Split data into training (80%) and testing (20%)
Step 6: Train Decision Tree model
Step 7: Train Random Forest model
Step 8: Evaluate models using classification metrics
Step 9: Compare performance

Workflow Diagram:



---

6. Performance Analysis

The following evaluation metrics were used:

Accuracy

$Accuracy = TP + TN \mid TP + TN + FP + FN$

Measures overall correctness.

Precision

$Precision = TP \mid TP + FP$

Measures correctness of positive predictions.

Recall

$Recall = TP \mid TP + FN$

Measures ability to detect actual positive cases.

F1 Score

$F1 = 2 \cdot Precision \cdot Recall \mid Precision + Recall$

Balances precision and recall.

Observations

- Decision Tree achieved moderate accuracy but showed slight overfitting.
- Random Forest achieved higher and more stable accuracy.
- Random Forest reduced variance due to ensemble averaging.
- Confusion matrix showed better balance in Random Forest predictions.

Random Forest generally performed better for stock movement classification.

---

7. Hyperparameter Tuning

Decision Tree Tuned Parameters:

- max_depth
- min_samples_split
- criterion (gini / entropy)

Increasing max_depth increases complexity but may cause overfitting.

Random Forest Tuned Parameters:

- n_estimators (number of trees)
- max_depth
- max_features

Increasing n_estimators improves stability but increases computation time.

GridSearchCV was used to find optimal parameters.

Optimal tuning improved test accuracy and reduced overfitting.

Output:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.preprocessing import StandardScaler
```

```
data = pd.read_csv('/content/ADANIPORTS.csv')

# Create Classification Target
data['Movement'] = np.where(data['Close'] > data['Open'], 1, 0)

data.head()
```

	Date	Symbol	Series	Prev Close	Open	High	Low	Last	Close	VWAP	Volume	Turnover	Trades	Deliverable Volume	%De
0	2007-11-27	MUNDRAPORT	EQ	440.00	770.00	1050.00	770.0	959.0	962.90	984.72	27294366	2.687719e+15	NaN	9859619	
1	2007-11-28	MUNDRAPORT	EQ	962.90	984.00	990.00	874.0	885.0	893.90	941.38	4581338	4.312765e+14	NaN	1453278	
2	2007-11-29	MUNDRAPORT	EQ	893.90	909.00	914.75	841.0	887.0	884.20	888.09	5124121	4.550658e+14	NaN	1069678	
3	2007-11-30	MUNDRAPORT	EQ	884.20	890.00	958.00	890.0	929.0	921.55	929.17	4609762	4.283257e+14	NaN	1260913	
4	2007-12-03	MUNDRAPORT	EQ	921.55	939.75	995.00	922.0	980.0	969.30	965.65	2977470	2.875200e+14	NaN	816123	

```python
X = data[['Open', 'High', 'Low', 'Volume']]
y = data['Movement']

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```python
dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train, y_train)

# Prediction
y_pred_dt = dt_model.predict(X_test)
```

[8]
0s

```python
print("Decision Tree Results")
print("Accuracy:", accuracy_score(y_test, y_pred_dt))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_dt))
print("Classification Report:\n", classification_report(y_test, y_pred_dt))
```

```
Decision Tree Results
Accuracy: 0.681203007518797
Confusion Matrix:
 [[238 115]
 [ 97 215]]
Classification Report:
               precision    recall  f1-score   support

           0       0.71      0.67      0.69       353
           1       0.65      0.69      0.67       312

    accuracy                           0.68       665
   macro avg       0.68      0.68      0.68       665
weighted avg       0.68      0.68      0.68       665
```

[9]
2s

```python
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)

# Prediction
y_pred_rf = rf_model.predict(X_test)
```

```python
print("Random Forest Results")
print("Accuracy:", accuracy_score(y_test, y_pred_rf))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_rf))
print("Classification Report:\n", classification_report(y_test, y_pred_rf))
```

```
Random Forest Results
Accuracy: 0.6947368421052632
Confusion Matrix:
 [[265  88]
 [115 197]]
Classification Report:
               precision    recall  f1-score   support

           0       0.70      0.75      0.72       353
           1       0.69      0.63      0.66       312

    accuracy                           0.69       665
   macro avg       0.69      0.69      0.69       665
weighted avg       0.69      0.69      0.69       665
```

```
[11]
models = ['Decision Tree', 'Random Forest']
accuracies = [
    accuracy_score(y_test, y_pred_dt),
    accuracy_score(y_test, y_pred_rf)
]

plt.figure()
plt.bar(models, accuracies)
plt.xlabel("Models")
plt.ylabel("Accuracy")
plt.title("Model Comparison")
plt.show()
```

Conclusion

In this experiment, Decision Tree and Random Forest algorithms were implemented to classify stock movement (Up/Down) using the ADANIPORTS dataset.

The Decision Tree model achieved an accuracy of **68.12%**, with balanced precision and recall values around 0.68. While the model was able to correctly classify a significant number of stock movements, it showed some misclassification due to overfitting and the inherent volatility of stock market data.

The Random Forest model achieved a slightly higher accuracy of **69.47%** and a better overall F1-score of approximately 0.69. By combining multiple decision trees, Random Forest reduced variance and improved prediction stability compared to the single Decision Tree model.

Overall, Random Forest outperformed Decision Tree in terms of accuracy and robustness. However, both models demonstrated moderate performance, highlighting the complexity and unpredictability of financial market data